

ZAČÍNÁME S HERNÍ KONZOLÍ PICOPAD

Rychlá příručka s popisem
a vzorovými kódy

Obsah:

1.	Specifikace	4
1.1	Picopad.....	4
1.2	Picopad Wi-Fi	6
1.3	Picopad Pro	8
2.	Instalace vývojového prostředí.....	10
3.	Pracujeme s knihovnou PicoLibSDK	13
3.1	Nástroje knihovny PicoLibSDK	15
3.2	Ověření funkčnosti prostředí.....	16
4.	Pracujeme s PlatformIO	17
4.1	Ověření funkčnosti prostředí.....	19
5.	Základní příkazy knihovny PicoLibSDK	20
5.1	Příkazy pro práci se zařízením	20
5.2	Příkazy pro práci s klávesami	20
5.3	Příkazy pro práci se stavovými LED diodami	21
5.4	Příkazy pro práci s displejem.....	22
5.5	Příkazy pro práci s grafikou.....	23
5.6	Příkazy pro práci se zvukovými soubory	35

Tato příručka Vás seznámí s herní konzolí [Picopad](#), jejím hardwarovým vybavením a popíše vzorové kódy, které Vás uvedou do základní problematiky programování.

Programování lze provádět dvěma způsoby. Buď pomocí kompilery v rámci [PicoLibSDK](#) nebo pomocí [PlatformIO](#) pro [Visual Studio Code](#). Pokusíme se ukázat oba způsoby, abyste si mohli vybrat ten, který Vám nejlépe vyhovuje.

Co budeme potřebovat:

HARDWARE:

- herní konzole [Picopad](#), [Picopad WiFi](#) nebo [Picopad Pro](#)
 - v tomto tutoriálu budeme používat verzi [Picopad Pro](#)
 - příručku lze použít pro všechny verze herní konzole, pouze části zabývající se připojením k Wi-Fi je možno vyzkoušet jen u verzí [Wi-Fi](#) a [Pro](#)
- [mikro USB kabel](#)
- základní elektronické součástky, např. rezistory, LED diody, potenciometr apod.

SOFTWARE:

- [ARM GCC Compiler](#)
- [Git](#)
- [Visual Studio Code](#) s nainstalovaným rozšířením [PlatformIO](#), [C/C++](#), [Batch Runner](#)

1. Specifikace

1.1 Picopad

Herní konzole [Picopad](#) je založena na jednočipovém počítači [Raspberry Pi Pico](#), který je postavený na mikročipu RP2040.



Specifikace:

- 2“ IPS displej s rozlišením 240x320 pixelů
- slot pro microSD kartu
- ovládací prvky v podobě osového kříže a čtyř ovládacích tlačítek
- přepínač POWER
- tlačítko Reset a BOOTSEL
- stavové LED diody
- reproduktor
- externí konektor
- baterie 500 mAh nabíjená přes modul s TP4056 (ochrana proti přepětí, podpětí a nadproudu)

Externí konektor:

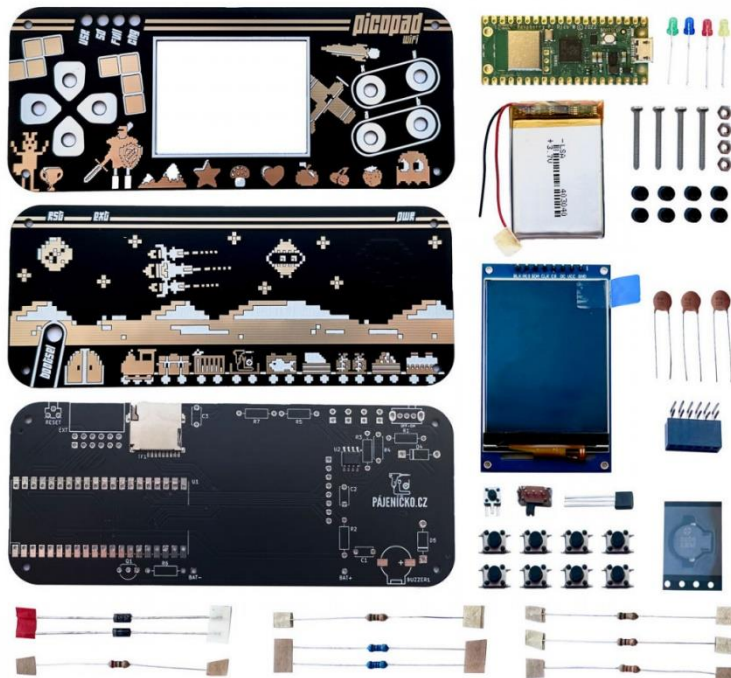


Příslušenství:

- [USB adaptér](#)
- [rámeček](#)

1.2 Picopad Wi-Fi

Herní konzole [Picopad Wi-Fi](#) je založena na jednočipovém počítači [Raspberry Pi Pico W](#), který je postavený na mikročipu RP2040.



Specifikace:

- 2“ IPS displej s rozlišením 240x320 pixelů
- slot pro microSD kartu
- ovládací prvky v podobě osového kříže a čtyř ovládacích tlačítek
- přepínač POWER
- tlačítko Reset a BOOTSEL
- stavové LED diody
- reproduktor
- externí konektor
- baterie 500 mAh nabíjená přes modul s TP4056 (ochrana proti přepětí, podpětí a nadproudu)

Konektivita:

- Wi-Fi 802.11n, 2.4 GHz, WPA 3
- Bluetooth 5.2

Externí konektor:



Příslušenství:

- [USB adaptér](#)
- [rámeček](#)

1.3 Picopad Pro

Herní konzole [Picopad Pro](#) je založena na jednočipovém počítači [Raspberry Pi Pico W](#), který je postavený na mikročipu RP2040.



Specifikace:

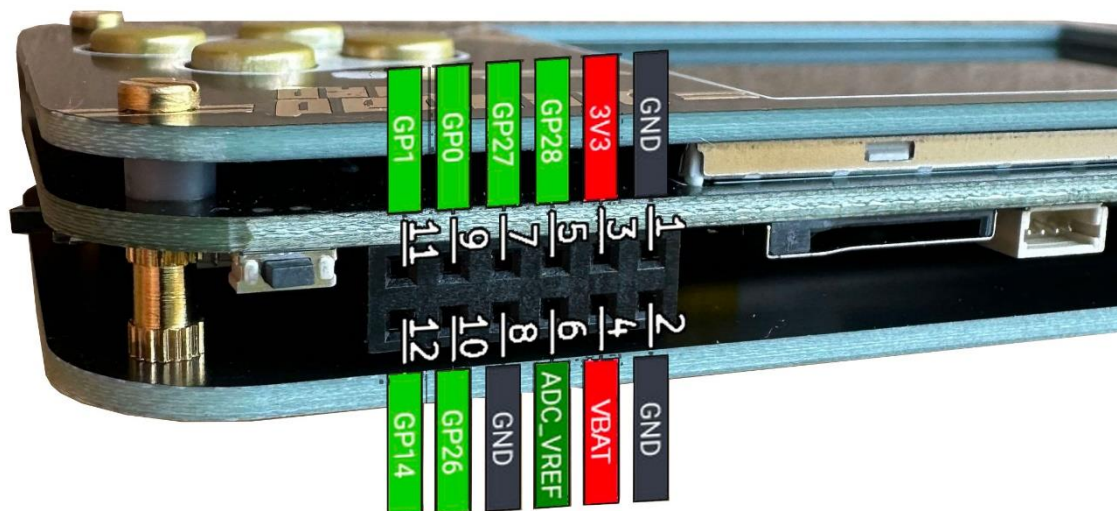
- 2,8“ IPS displej s rozlišením 240x320 pixelů
- slot pro microSD kartu
- ovládací prvky v podobě osového kříže a čtyř ovládacích tlačítek
- přepínač POWER a MUTE
- tlačítko Reset a BOOTSEL
- stavové LED diody
- reproduktor
- 3,5 mm konektor pro sluchátka
- sběrnice Picobus
- externí konektor
- I2C konektor
- baterie 600 mAh, nabíjena přes modul s TP4056 (ochrana proti přepětí, podpětí a nadproudu)

Konektivita:

- Wi-Fi 802.11n, 2.4 GHz, WPA 3
- Bluetooth 5.2

Externí konektor:

U této verze si dejte pozor na zapojení externího konektoru, oproti původní verzi je otočen o 180 °.



Příslušenství:

- [videokarta](#)
- [USB adaptér](#)
- [sada externí baterie](#)
- [rámeček](#)

2. Instalace vývojového prostředí

V první řadě si stáhneme potřebnou softwarovou výbavu:

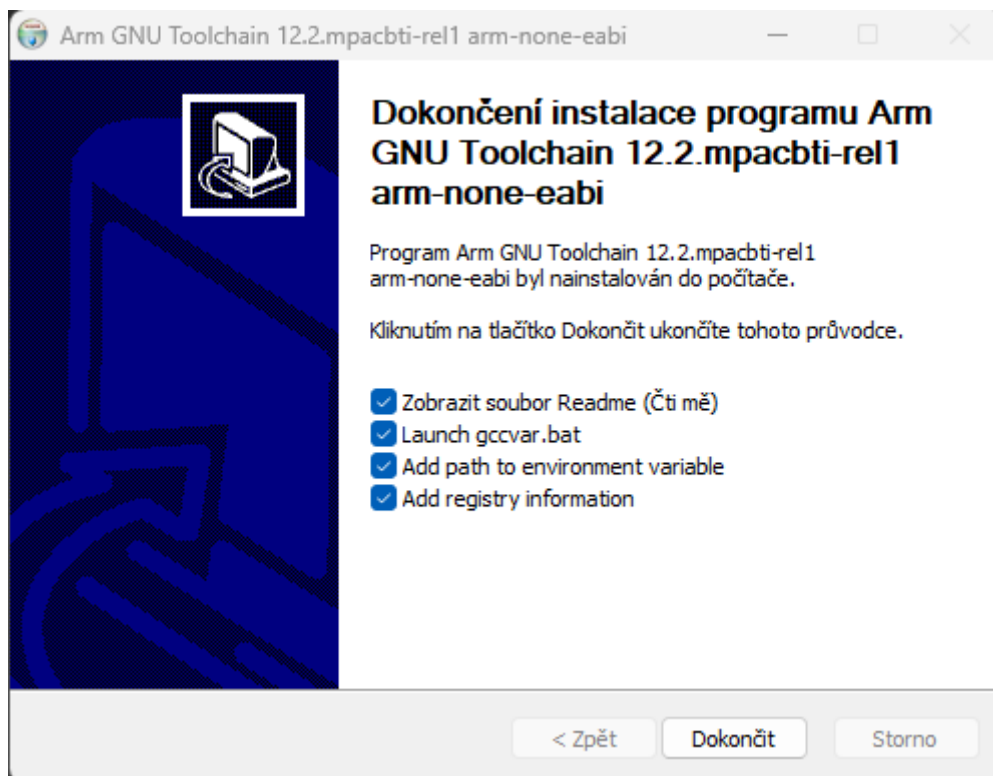
- [ARM GCC Compiler](#)
- [git](#)
- [Visual Studio Code](#)

Kompilátory spolu s programy vývojového prostředí doporučujeme instalovat buď přímo na systémový disk nebo do jednoho adresáře, např. C:\development. Instalace do hlubší adresářové struktury by mohlo vést k nesprávnému chování vývojového prostředí. Pro repositáře a knihovny lze vytvořit např. adresář sdk v cestě C:\development. Zachováme tím větší přehlednost celého prostředí.

Máme-li vše staženo, začneme instalací:

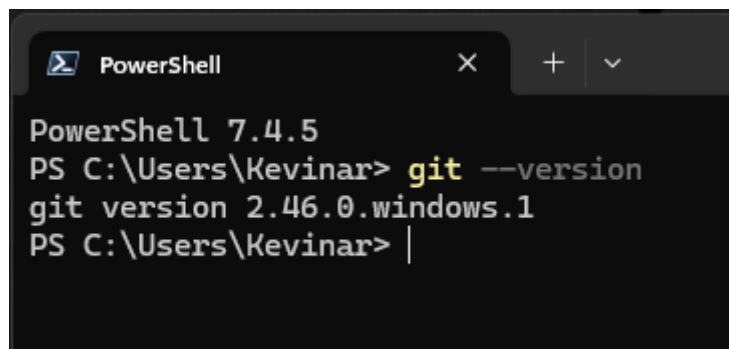
1) [ARM GCC Compiler](#):

- během instalace nezapomeňte zaškrtnout možnost *Add path to environment variable*
- zkontrolujte přidání C:\Development\arm-none-eabi\bin do uživatelských proměnných PATH (cestu upravte dle svých potřeb)
- ověření správné instalace provedete z příkazového řádku zadáním příkazu *arm-none-eabi-gcc --version*



2) [Git](#):

- při výběru komponent zkontrolujte zaškrtnutí u *Check daily for Git for Windows Update, (NEW!) Add a Git bash Profile to Windows Terminal a (NEW!) Scalar (Git add-on to manage large- scale repositories)*
- jako výchozí editor Git zvolte vámi preferovanou možnost
- u výběru názvů pro nové branch doporučujeme nastavit možnost *Override the default branch name for new repositories*
- pro nastavení PATH prostředí nechte předvolenou volbu *Git from the command line and also from 3-rd party software*
- SSH klienta zvolte dle svých preferencí, stejně tak v dalším kroku knihovnu SSL/TSL
- konfiguraci převodu konců řádku ponechte v systémech Windows ve výchozím nastavení *Checkout Windows-style, commit Unix-style line ending*
- emulátor terminálu ponechte ve výchozím nastavení *Use MinTTY (the default terminal of MSYS2)*
- chování příkazu 'git pull' je doporučeno nastavit na *Fast-forward or merge*
- správce hesel nechte nastaven na *Git Credencial Manager*
- při výběru extra možností zkontrolujte zaškrtnutí volby *Enable file system caching*
- u experimentálních možností nezaškrťvejte žádnou volbu
- po dokončení instalace ověřte její správný průběh příkazem `git --version`



```
PowerShell 7.4.5
PS C:\Users\Kevinar> git --version
git version 2.46.0.windows.1
PS C:\Users\Kevinar> |
```

- příkazem `git config --system core.longpaths true` povolte dlouhé cesty k souborům
- restartujte počítač

3) [Visual Studio Code](#):

- instaluje se jako běžný program
- nainstalujte rozšíření [PlatformIO](#)
- nainstalujte rozšíření [C/C++](#)
- nainstalujete rozšíření [Batch Runner](#), umožňuje spouštět dávkové soubory v terminálu [Visual Studio Code](#)

4) Otevřeme PowerShell a zadáme příkaz

```
winget install Microsoft.VisualStudio.2022.BuildTools --force --override "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools --add Microsoft.VisualStudio.Component.VC.Tools.x86.x64 --add Microsoft.VisualStudio.Component.Windows11SDK.22000"
```

Příkazem spustíme instalaci Microsoft Visual Studio Tools. Ten je potřeba pro správnou funkci IntelliSense ve [Visual Studio Code](#).

5) Klonování potřebných úložišť:

a) pico-sdk

Pro tento repositář doporučujeme vytvořit adresář pico, do kterého budeme klonovat. Přejděte do něj, spusťte PowerShell a zadejte příkaz

```
git clone --recurse-submodules https://github.com/raspberrypi/pico-sdk.git
```

Přejděte do nově vytvořeného adresáře pico-sdk příkazem `cd pico-sdk` a proveďte kontrolu všech stažených submodulů pomocí

```
git submodule update --init --recursive
```

Abychom nemuseli knihovnu kopírovat do každého nového projektu, nastavíme její cestu do proměnného prostředí PATH ve Windows. Spusťte PowerShell a zadejte příkaz

```
setx PICO_SDK_PATH "C:\development\sdk\pico\pico-sdk"
```

Cestu ke knihovně Pico SDK samozřejmě upravte podle své potřeby.

b) PicoLibSDK

Přejděte do adresáře, kam chcete repositář klonovat, spusťte PowerShell a zadejte příkaz

```
git clone --recurse-submodules https://github.com/Panda381/PicoLibSDK.git
```

c) Picopad

Přejděte do adresáře, kam chcete repositář klonovat, spusťte PowerShell a zadejte příkaz

```
git clone --recurse-submodules https://github.com/Pajenicko/Picopad.git
```

Nyní máme naklonovány všechny repositáře, které budeme potřebovat.

3. Pracujeme s knihovnou PicoLibSDK

V knihovně [PicoLibSDK](#) najdete několik adresářů, které mají tento význam:

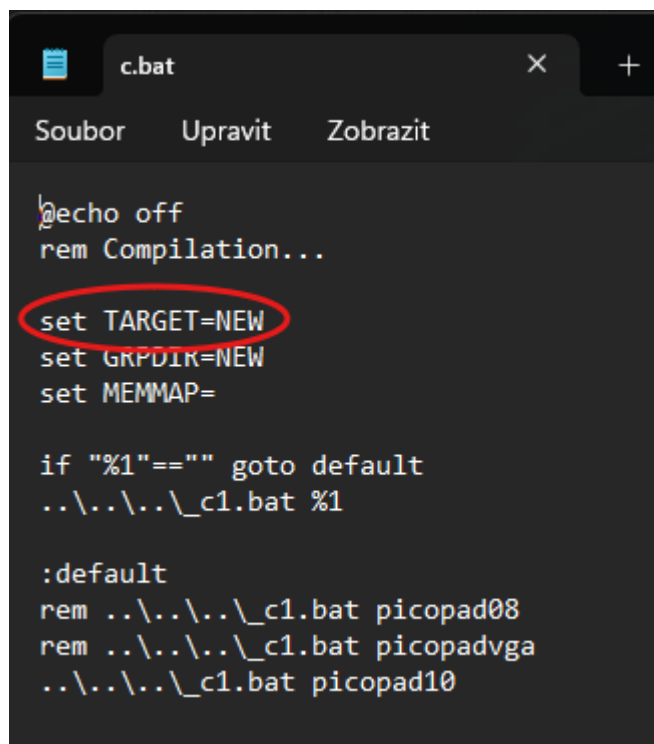
<code>!<název adresáře></code>	kompilované ukázkové programy
<code>_<název adresáře></code>	interní soubory knihovny
<code><název adresáře></code>	zdrojové kódy ukázkových programů

Adresář v cestě `PicoLibSDK\PicoPad\NEW\NEW` je tzv. vzorový a slouží pro vytváření nových projektů. Pokud budeme chtít začít s novým projektem, stačí vytvořit její kopii a pojmenovat ji dle potřeby.

Uvnitř adresáře nalezneme několik dávkových souborů:

<code>c.bat</code>	kompilace programu
<code>d.bat</code>	smazání dočasných souborů kompilace
<code>e.bat</code>	odeslání kompilovaného programu do zařízení
<code>a.bat</code>	provede všechny předešlé popsané činnosti

Před samotnou kompilací nového programu je potřeba upravit název projektu i v samotných dávkových souborech. Stačí je otevřít v jakémkoliv editačním programu (např. Poznámkový blok) a upravit část za rovnítkem u položky `set TARGET`. Název projektu nesmí obsahovat více jak 8 znaků.



```
c.bat
Soubor  Upravit  Zobrazit

echo off
rem Compilation...
set TARGET=NEW
set GRPDIR=NEW
set MEMMAP=

if "%1"==" " goto default
..\..\..\_c1.bat %1

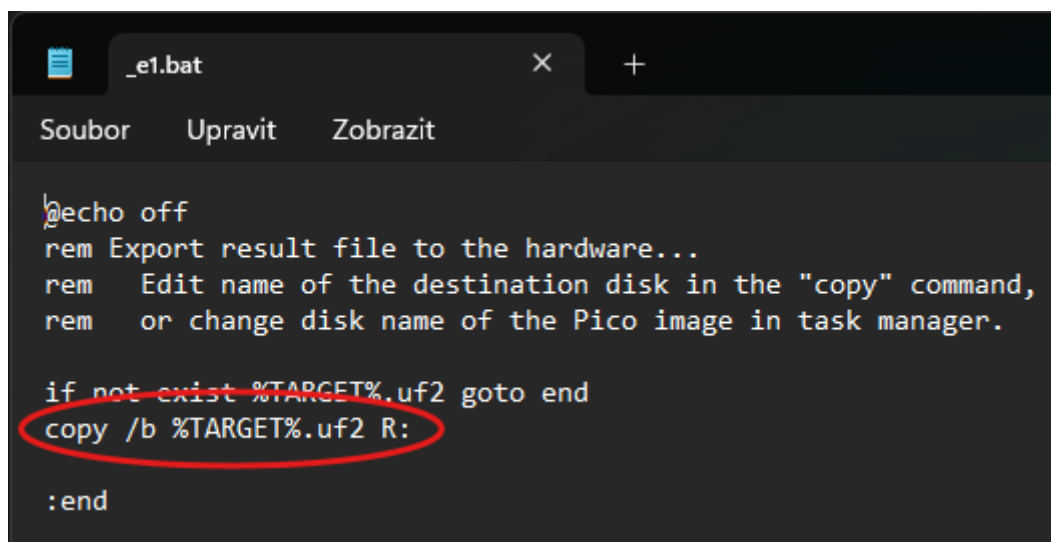
:default
rem ..\..\..\_c1.bat picopad08
rem ..\..\..\_c1.bat picopadvga
..\..\..\_c1.bat picopad10
```

Dále uvnitř adresáře najdeme složku *src*. V ní jsou vloženy soubory *main.cpp* a *main.h*, soubor zdrojového kódu a hlavičkový soubor. Do těchto souborů budeme zapisovat zdrojový C/C++ kód programu.

Nahrání kompilovaného programu do herní konzole Picopad:

V režimu BOOTSEL se herní konzole připojuje k počítači jako USB Mass Storage. K nahrání programu stačí kompilovaný soubor *.UF2 na toto úložiště kopírovat. Po zkopírování se herní konzole restartuje a dojde k nahrání programu. Ten následně spustíme stiskem klávesy Y.

Pro automatizaci procesu kopírování slouží dávkový soubor *_e1.bat*, který se nachází v kořenovém adresáři [PicoLibSDK](#). V něm je přednastaven parametr kopírování pomocí příkazu *copy /b %TARGET%.uf2 R:*, kde R je systémem Windows přiřazené písmeno disku. Upravte písmeno disku dle vašich potřeb.

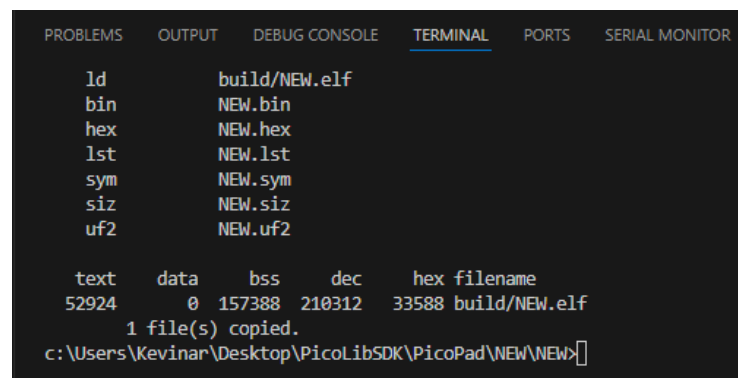


```
_e1.bat
Soubor  Upravit  Zobrazit

@echo off
rem Export result file to the hardware...
rem  Edit name of the destination disk in the "copy" command,
rem  or change disk name of the Pico image in task manager.

if not exist %TARGET%.uf2 goto end
copy /b %TARGET%.uf2 R:

:end
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SERIAL MONITOR

ld        build/NEW.elf
bin        NEW.bin
hex        NEW.hex
lst        NEW.lst
sym        NEW.sym
siz        NEW.siz
uf2        NEW.uf2

text      data    bss    dec    hex filename
52924     0 157388 210312 33588 build/NEW.elf
1 file(s) copied.
c:\Users\Kevinar\Desktop\PicoLibSDK\PicoPad\NEW\NEW>
```

3.1 Nástroje knihovny PicoLibSDK

Nástroje knihovny nalezneme ve složce `_tools`. Pro činnosti spjaté s herní konzolí [Picopad](#) nás budou zajímat především nástroje:

<i>PicoPadImg</i>	převod *.bmp obrázků do C pole
<i>RaspPicoSnd</i>	převod zvukových souborů do C pole

PicoPadImg:

- vstupní soubor musí být ve formátu Windows BMP s bitovou hloubkou 4-bit, 8-bit nebo 24-bit
- konverze se spouští dávkovým souborem *test.bat*, který je potřeba před spuštěním editovat dle potřeb

Generování nekomprimovaného obrázku:

PicoPadImg input.bmp output.c name

Generování komprimovaného RLE obrázku:

PicoPadImg input.bmp output.c name r

RaspPicoSnd:

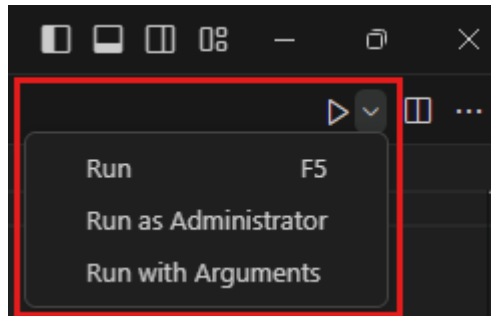
- vstupní soubor musí být ve formátu PCM, mono, 8-bit unsigned, vzorkovací frekvence 22 050 Hz nebo komprimovaný IMA ADPCM, vzorkovací frekvence 22 050 Hz
- konverze se spouští dávkovým souborem *test.bat*, který je před spuštěním potřeba editovat podle svých potřeb

Generování zvukového pole:

RaspPicoSnd input.bmp output.c name

3.2 Ověření funkčnosti prostředí

- 1) Připojte [Picopad](#) k počítači v režimu BOOTSEL
- 2) Otevřete složku *NEW* v editoru zdrojového kódu [Visual Studio Code](#)
Pokud jste již nainstalovali rozšíření [Batch Runner](#), měli byste po označení jednoho z dávkových souborů v pravém horním rohu vidět ikonu "Play"



- 3) Po levé straně, v stromové struktuře souborů a adresářů, označte dávkový soubor s názvem *a.bat*
- 4) Klikněte na ikonu "Play"

Provede se kompilace programu s následným nahráním do konzole [Picopad](#). Program nyní můžete spustit stiskem klávesy Y.

4. Pracujeme s PlatformIO

[PlatformIO](#) je multiplatformní nástroj určený pro vývoj softwaru embedded systémů a instaluje se jako rozšíření editoru zdrojového kódu [Visual Studio Code](#).

Pomocí [PlatformIO](#) můžeme [Picopad](#) programovat v Arduino kódu. Pro lepší pochopení práce s [PlatformIO](#) doporučujeme pročíst příručku [PlatformIO for VSCode](#).

Vzorový projekt s názvem *template* nalezneme v dříve naklonovaném repositáři v cestě `Picopad\arduino\template`. Projekt obsahuje kód Hello World a je připraven k okamžité kompilaci.

Před samotnou kompilací si popíšeme adresářovou strukturu projektu:

<i>assets</i>	soubory v něm obsažené budou po kompilaci vloženy do adresáře <i>build</i>
<i>build</i>	po kompilaci obsahuje *.UF2 soubor a soubory z adresáře <i>assets</i>
<i>include</i>	hlavičkové soubory projektu
<i>lib</i>	knihovny projektu
<i>patches</i>	specifické patch soubory pro Picopad
<i>src</i>	zdrojový kód programu
<i>tools</i>	specifické nástroje potřebné ke kompilaci programu

Pro nastavení vývojového prostředí projektu se používá soubor [platformio.ini](#). Ten lze použít ke konfiguraci více platforem a architektur najednou.

Vzorový obsah souboru platformio.ini:

```
[env]
name = TEMPLATE
platform = https://github.com/maxgerhardt/platform-raspberrypi.git
board = rpipicow
framework = arduino
board_build.core = earlephilhower
board_build.filesystem_size = 0m
monitor_speed = 115200

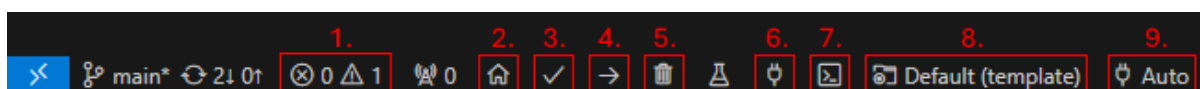
[env:picopad]
extra_scripts =
    pre:tools/patch_memmap.py
    post:tools/copy_build.py

[env:pico]
extra_scripts =
    pre:tools/revert_patch_memmap.py
    post:tools/copy_build.py
```

Pokud máte herní konzoly verze [Picopad](#), budete mít vždy nakonfigurovaný typ vývojové desky na *pico* (Raspberry Pi Pico). Používáte-li herní konzoly verze [Wi-Fi](#) nebo [Pro](#), můžete nakonfigurovat typ vývojové desky na *rpipicow* (Raspberry Pi Pico W). Poté budete moci ve svých projektech využívat i Wi-Fi připojení.

Chcete-li ve svém projektu použít externí knihovnu, můžete ji do projektu zahrnout nastavením závislosti pomocí parametru [lib_deps](#). Je-li závislost společná pro všechna vývojová prostředí konfiguruje ji v sekci `[env]`. Je-li závislost potřebná pouze v jednom z několika vývojových prostředí, konfiguruje ji v sekci `[env]` daného vývojového prostředí např: `[env:pycopad]`.

Ovládací prvky rozhraní:



- | | |
|--|---------------------------------------|
| 1. zobrazení počtu chyb či varování | |
| 2. PlatformIO: Home | zobrazení domovské stránky PlatformIO |
| 3. PlatformIO: Build | kompilace programu |
| 4. PlatformIO: Upload | nahrání programu do zařízení |
| 5. PlatformIO: Clean | vyčištění kompilačních souborů |
| 6. Serial Port Monitor | otevře Seriál Port Monitor |
| 7. PlatformIO: New Terminal | otevře nový terminál |
| 8. Switch PlatformIO Project Environment | výběr vývojového prostředí |
| 9. Set upload/monitor/test port | nastavení COM portu zařízení |

Tlačítka toolbaru je možno skrýt/zobrazit pomocí kontextového menu vyvolaného stiskem pravého tlačítka myši. Zároveň je možno jejich funkci [editovat](#) v nastavení rozšíření, čímž je umožněno vytvořit toolbar vlastní.

Nahrání kompilovaného programu do herní konzole Picopad:

V režimu BOOTSEL se herní konzole připojuje k počítači jako USB Mass Storage. K nahrání programu stačí kompilovaný soubor *.UF2 na toto úložiště kopírovat. Po zkopírování se herní konzole restartuje a dojde k nahrání programu. Ten následně spustíme stiskem klávesy Y.

Pro kopírování kompilovaného programu z prostředí [Visual Studio Code](#) použijeme příkazu [copy](#). Zjistíme, jaké je v systému Windows velkokapacitnímu médiu přiřazeno písmeno disku a poté do terminálu zadáme příkaz

```
copy <cesta k souboru>\*.uf2 <jednotka>:\
```

kde:

<cesta k souboru>

cesta ke kompilovanému souboru *.UF2

<jednotka>

systémem Windows přiřazené písmeno disku

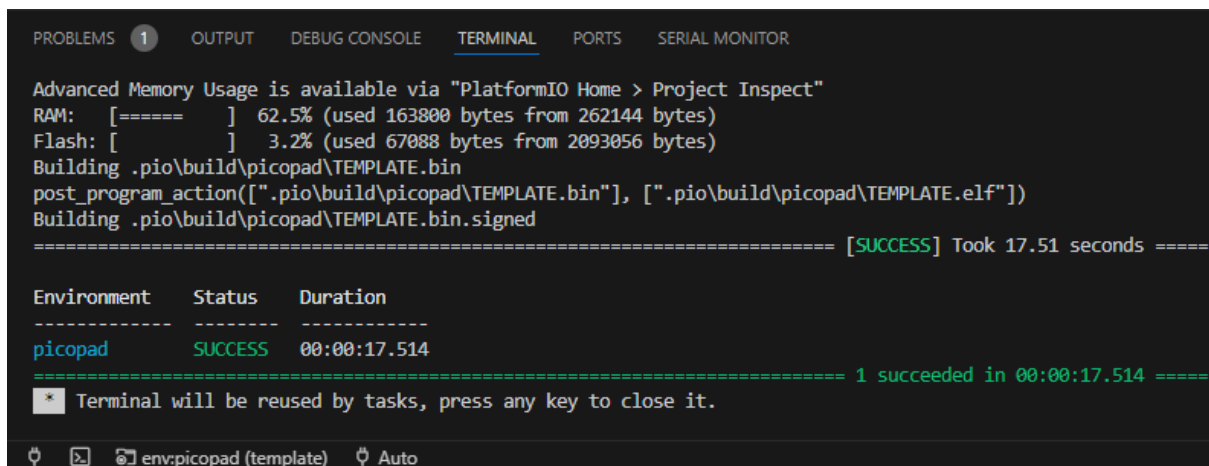
příklad tvaru příkazu

```
copy build\*.uf2 J:\
```

V případě, že bychom před dalším nahráním programu zadávali do terminálu příkazy jiné, není potřeba příkaz *copy* znovu psát, lze jej "nalistovat" pomocí kurzorových šipek nahoru a dolů.

4.1 Ověření funkčnosti prostředí

- 1) Otevřete složku *template* v editoru zdrojového kódu [Visual Studio Code](#)
- 2) Zvolíme vývojové prostředí projektu *env:picopad*
- 3) Stiskneme tlačítko *Build* k provedení kompilace programu



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [===== ] 62.5% (used 163800 bytes from 262144 bytes)
Flash: [===== ] 3.2% (used 67088 bytes from 2093056 bytes)
Building .pio\build\picopad\TEMPLATE.bin
post_program_action([".pio\build\picopad\TEMPLATE.bin"], [".pio\build\picopad\TEMPLATE.elf"])
Building .pio\build\picopad\TEMPLATE.bin.signed
===== [SUCCESS] Took 17.51 seconds =====

Environment   Status      Duration
-----
picopad       SUCCESS     00:00:17.514
===== 1 succeeded in 00:00:17.514 =====

* Terminal will be reused by tasks, press any key to close it.
```

- 4) Připojíme herní konzoli [Picopad](#) k počítači v režimu BOOTSEL
- 5) V panelu rozhraní se přepneme do terminálu (*pwsh*) a spustíme příkaz pro kopírování souboru *copy build/*uf.2 J:*
- 6) Provede se zkopírování programu do konzole [Picopad](#). Program nyní můžete spustit stiskem klávesy Y.

5. Základní příkazy knihovny PicoLibSDK

Podrobný popis všech příkazů je možno nalézt v originální dokumentaci k [PicoLibSDK](#).

5.1 Příkazy pro práci se zařízením

Pro inicializaci zařízení je využíván hlavičkový soubor *picopad_init.h*, který nalezneme ve složce *_devices/picopad*.

DeviceInit();

- inicializace zařízení

DeviceTerm();

- ukončení všech běžících procesů a uvolnění zdrojů zařízení

5.2 Příkazy pro práci s klávesami

Pro práci s klávesami zařízení je využíván hlavičkový soubor *picopad_key.h*, který nalezneme ve složce *_devices/picopad*.

KeyInit();

- inicializace systému obsluhy kláves

KeyTerm();

- ukončení všech běžících procesů a uvolnění zdrojů systému obsluhy kláves

KeyPressed(u8 key);

- kontrola stisku specifikované klávesy
- návratová hodnota *True* pro zjištění stisknuté klávesy
- návratová hodnota *False* pro zjištění, že klávesa není stisknuta

u8 key

definice kontrolované klávesy

KeyScan();

- skenování stisknutých kláves prostřednictvím přerušení časovače (SysTick), prováděno pravidelně, aktualizuje stav stisknutých kláves

KeyGet();

- zjištění stisknuté klávesy
- návratová hodnota je *kód klávesy* nebo *NOKEY*

KeyChar();

- získání znaku stisknuté klávesy
- návratová hodnota je znak klávesnice nebo *NOCHAR*

KeyFlush();

- vyprázdní bufferu systému pro zpracování stisku kláves

KeyRet(u8 key);

- vložení specifikované klávesy do bufferu systému pro zpracování stisku kláves
je-li buffer plný dojde k jeho přepsání

KeyNoPressed();

- kontroluje, zda není stisknuta žádná klávesnice
- návratová hodnota *True* není-li stisknuta žádná klávesa
- návratová hodnota *False* je-li stisknuta jakákoliv klávesa

KeyWaitNoPressed();

- vyčkání, dokud není stisknuta žádná klávesa

5.3 Příkazy pro práci se stavovými LED diodami

Pro práci s LED diodami je využíván hlavičkový soubor *picopad_led.h*, který nalezneme ve složce *_devices/picopad*.

Definice indexů LED diod:

LED1	0	žlutá LED dioda USB
LED2	1	zelená LED na desce mikrokontroleru

LedOn(u8 inx);

- zapnutí specifikované LED diody

u8 inx definice indexu LED diody

LedOff(u8 inx);

- vypnutí specifikované LED diody

u8 inx

definice indexu LED diody

LedFlip(u8 inx);

- přepnutí stavu specifikované LED diody

u8 inx

definice indexu LED diody

LedInit();

- inicializace systému obsluhujícího funkce LED diod

LedTerm();

- ukončení všech běžících procesů a uvolnění zdrojů systému obsluhy LED diod

5.4 Příkazy pro práci s displejem

Pro práci s displejem je využívám hlavičkový soubor `st7789.h`, který nalezneme ve složce `_display/st7789`.

DispRotation(u8 rot);

- nastavení rotace displeje, kde *rot* může nabývat hodnoty

0	portrait
1	landscape
2	inverted portrait
3	inverted landscape

DispSetModeFlipX(Bool flipx);

- horizontální převrácení obrázku

DispSetModeFlipY(Bool flipy);

- vertikální převrácení obrázku

DispUpdate();

- aktualizace zobrazení displeje

5.5 Příkazy pro práci s grafikou

Pro vykreslování grafických objektů a práci s grafikou displeje je využíván hlavičkový soubor `lib_draw.h`, který nalezneme ve složce `_lib/inc` pro [PicoLibSDK](#)

Příkazy pro definování základních RGB barev:

COL_BLACK	černá	RGB	0, 0, 0
COL_BLUE	modrá	RGB	0, 0, 255
COL_GREEN	zelená	RGB	0, 255, 0
COL_CYAN	azurová	RGB	0, 255, 255
COL_RED	červená	RGB	255, 0, 0
COL_MAGENTA	purpurová	RGB	255, 0, 255
COL_YELLOW	žlutá	RGB	255, 255, 0
COL_WHITE	bílá	RGB	255, 255, 255
COL_GRAY	šedá	RGB	128, 128, 128
COL_AZURE	azurová	RGB	0, 127, 255
COL_ORANGE	oranžová	RGB	255, 127, 0
COL_DKBLUE	tmavě modrá	RGB	0, 0, 127
COL_DKGREEN	tmavě zelená	RGB	0, 127, 0
COL_DKCYAN	tmavě azurová	RGB	0, 127, 127
COL_DKRED	tmavě červená	RGB	127, 0, 0
COL_DKMAGENTA	tmavě purpurová	RGB	127, 0, 127
COL_DKYELLOW	tmavě žlutá	RGB	127, 127, 0
COL_DKWHITE	tmavě bílá	RGB	127, 127, 127
COL_DKGRAY	tmavě šedá	RGB	64, 64, 64
COL_LTBLUE	světle modrá	RGB	127, 127, 255
COL_LTGREEN	světle zelená	RGB	127, 255, 127
COL_LTCYAN	světle azurová	RGB	127, 255, 255
COL_LTRD	světle červená	RGB	255, 127, 127
COL_LTMAGENTA	světle purpurová	RGB	255, 127, 255
COL_LTYELLOW	světle žlutá	RGB	255, 255, 127
COL_LTGRAY	světle šedá	RGB	192, 192, 192

Příkazy pro definování typu písma:

SetFont5x8();	písmo 5x8 px
SetFont6x8();	písmo 6x8 px
SetFont8x8();	písmo 8x8 px
SetFont8x14();	písmo 8x14 px
SetFont8x16();	písmo 8x16 px

DrawRect(int x, int y, int w, int h, COLTYPE col);

- vykreslení obdélníku s těmito parametry

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
COLTYPE col	definice barvy výplně

DrawRectInv(int x, int y, int w, int h);

- vykreslení inverzního obdélníku s těmito parametry

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce

DrawFrame(int x, int y, int w, int h, COLTYPE col);

- vykreslení obdélníku bez výplně s těmito parametry

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
COLTYPE col	definice barvy obrazce

DrawFrameW(int x, int y, int w, int h, COLTYPE col, int width);

- vykreslení obdélníku bez výplně s těmito parametry

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
COLTYPE col	definice barvy obrazce
int width	šířka obvodové čáry obrazce

DrawFrameInv(int x, int y, int w, int h);

- vykreslení inverzního obdélníku bez výplně s těmito parametry

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce

DrawFrameInvW(int x, int y, int w, int h, int width);

- vykreslení inverzního obdélníku bez výplně s těmito parametry

int x	počáteční pozice na ose x pro levý horní roh obrazce
int y	počáteční pozice na ose y pro levý horní roh obrazce
int w	šířka obrazce
int h	výška obrazce
int width	šířka obvodové čáry obrazce

DrawClearCol(COLTYPE col);

- vyplní displej definovanou barvou

COLTYPE col	definice barvy výplně
-------------	-----------------------

DrawClear();

- vyčistění displeje (vyplnění černou barvou)

DrawPoint(int x, int y, COLTYPE col);

- vykreslení bodu s těmito parametry

int x	pozice bodu na ose x
int y	pozice bodu na ose y
COLTYPE col	definice barvy bodu

DrawPointInv(int x, int y);

- vykreslení invertního bodu s těmito parametry

int x	pozice bodu na ose x
int y	pozice bodu na ose y

DrawLineClip(int x1, int y1, int x2, int y2, COLTYPE col, int xmin, int xmax, int ymin, int ymax);

- vykreslení úsečky v rozsahu definované oblasti dané parametry

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
COLTYPE col	definice barvy úsečky
int xmin	pozice na ose x pro levou horní hranici oblasti oříznutí úsečky
int ymin	pozice na ose y pro levou horní hranici oblasti oříznutí úsečky
int xmax	pozice na ose x pro pravou dolní hranici oblasti oříznutí úsečky
int ymax	pozice na ose y pro pravou dolní hranici oblasti oříznutí úsečky

DrawLine(int x1, int y1, int x2, int y2, COLTYPE col);

- vykreslení úsečky s těmito parametry

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
COLTYPE col	definice barvy úsečky

DrawLineW(int x1, int y1, int x2, int y2, COLTYPE col, int w, Bool round);

- vykreslení úsečky s těmito parametry

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
COLTYPE col	definice barvy úsečky
int w	šířka úsečky
Bool round	poloměr zaoblení koncových hran úsečky

DrawLineInvClip(int x1, int y1, int x2, int y2, int xmin, int xmax, int ymin, int ymax);

- vykreslení invertní úsečky v rozsahu definované oblasti dané parametry

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
int xmin	pozice na ose x pro levou horní hranici oblasti oříznutí úsečky
int ymin	pozice na ose y pro levou horní hranici oblasti oříznutí úsečky
int xmax	pozice na ose x pro pravou dolní hranici oblasti oříznutí úsečky
int ymax	pozice na ose y pro pravou dolní hranici oblasti oříznutí úsečky

DrawLineInv(int x1, int y1, int x2, int y2);

- vykreslení invertní úsečky s těmito parametry

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y

DrawLineInvW(int x1, int y1, int x2, int y2, int w, Bool round);

- vykreslení invertní úsečky s těmito parametry

int x1	pozice počátečního bodu úsečky na ose x
int y1	pozice počátečního bodu úsečky na ose y
int x2	pozice koncového bodu úsečky na ose x
int y2	pozice koncového bodu úsečky na ose y
int w	šířka úsečky
Bool round	poloměr zaoblení koncových hran úsečky

DrawFillCircle(int x0, int y0, int r, COLTYPE col);

- vykreslí kružnici s výplní s těmito parametry

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice
COLTYPE col	definice barvy výplně obrazce

DrawFillCircleInv(int x0, int y0, int r);

- vykreslí invertní kružnici s výplní s těmito parametry

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice

DrawCircle(int x0, int y0, int r, COLTYPE col);

- vykreslí kružnici s těmito parametry

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice
COLTYPE col	definice barvy obrazce

DrawCircleInv(int x0, int y0, int r);

- vykreslí invertní kružnici s těmito parametry

int x0	pozice středového bodu kružnice na ose x
int y0	pozice středového bodu kružnice na ose y
int r	poloměr kružnice

DrawRing(int x0, int y0, int rin, int rout, COLTYPE col);

- vykreslí prstenec s těmito parametry

int x0	pozice středového bodu prstence na ose x
int y0	pozice středového bodu prstence na ose y
int rin	poloměr vnitřní kružnice prstence
int rout	poloměr vnější kružnice prstence
COLTYPE col	definice barvy obrazce

DrawRingInv(int x0, int y0, int rin, int rout);

- vykreslí invertní prstenec s těmito parametry

int x0	pozice středového bodu prstence na ose x
int y0	pozice středového bodu prstence na ose y
int rin	poloměr vnitřní kružnice prstence
int rout	poloměr vnější kružnice prstence

DrawChar(char ch, int x, int y, COLTYPE col);

- vykreslí znak s transparentním pozadím dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawCharH(char ch, int x, int y, COLTYPE col);

- vykreslí znak s dvojnásobnou výškou a transparentním pozadím dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawCharW(char ch, int x, int y, COLTYPE col);

- vykreslí jeden znak s dvojnásobnou šířkou a transparentním pozadím dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawChar2(char ch, int x, int y, COLTYPE col);

- vykreslí jeden znak o dvojnásobné velikosti s transparentním pozadím dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku

DrawCharBg(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak s barvou pozadí dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBgH(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak s dvojnásobnou výškou a barvou pozadí dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBgW(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak s dvojnásobnou šířkou a barvou pozadí dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBg2(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak o dvojnásobné velikosti s barvou pozadí dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawCharBg4(char ch, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí jeden znak o čtyřnásobné velikosti s barvou pozadí dle těchto parametrů

char ch	definice znaku
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawText(const char* text, int x, int y, COLTYPE col);

- vykreslí text s transparentním pozadím dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawTextH(const char* text, int x, int y, COLTYPE col);

- vykreslí text s dvojnásobnou výškou a transparentním pozadím dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawTextW(const char* text, int x, int y, COLTYPE col);

- vykreslí text s dvojnásobnou šířkou a transparentním pozadím dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawText2(const char* text, int x, int y, COLTYPE col);

- vykreslí text o dvojnásobné velikosti s transparentním pozadím dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu

DrawTextBg(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text s barvou pozadí dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBgH(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text s dvojnásobnou výškou a barvou pozadí dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBgW(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text s dvojnásobnou šířkou a barvou pozadí dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBg2(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text o dvojnásobné velikosti s barvou pozadí dle těchto parametrů

const char* text	definice textu
int x	pozice počátečního znaku textu na ose x
int y	pozice počátečního znaku textu na ose y
COLTYPE col	definice barvy textu
COLTYPE bgcol	definice barvy pozadí

DrawTextBg4(const char* text, int x, int y, COLTYPE col, COLTYPE bgcol);

- vykreslí text o čtyřnásobné velikosti s barvou pozadí dle těchto parametrů

const char* text	definice textu
int x	pozice znaku na ose x
int y	pozice znaku na ose y
COLTYPE col	definice barvy znaku
COLTYPE bgcol	definice barvy pozadí

DrawImg(const COLTYPE* src, int xs, int ys, int xd, int yd, int w, int h, int ws);

- vykreslí obrázek či jeho část na displeji dle těchto parametrů

const COLTYPE* src	ukazatel na zdrojový bitmapový obraz
int xs	pozice počátku čtení zdrojového obrázku na ose x
int ys	pozice počátku čtení zdrojového obrázku na ose y
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku
int ws	šířka obrázku uložená do paměti

DrawImgPal(const u8* src, const u16* pal, int xs, int ys, int xd, int yd, int w, int h, int ws);

- vykreslí obrázek či jeho část na displeji dle těchto parametrů

const u8* src	ukazatel na zdrojový bitmapový obrázek
const u16* pal	ukazatel na paletu barev zdrojového bitmapového obrázku
int xs	pozice počátku čtení zdrojového obrázku na ose x
int ys	pozice počátku čtení zdrojového obrázku na ose y
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku
int ws	šířka obrázku uložená do paměti

DrawImg4Pal(const u8* src, const u16* pal, int xs, int ys, int xd, int yd, int w, int h, int ws);

- vykreslí 4bitový paletový obrázek či jeho část na displeji dle těchto parametrů

const u8* src	ukazatel na zdrojový bitmapový obrázek
const u16* pal	ukazatel na paletu barev zdrojového bitmapového obrázku
int xs	pozice počátku čtení zdrojového obrázku na ose x
int ys	pozice počátku čtení zdrojového obrázku na ose y
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku
int ws	šířka obrázku uložená do paměti

DrawImg4Rle(const u8* src, const u16* pal, int xd, int yd, int w, int h);

- vykreslí celý 4bitový obrázek v RLE formátu dle těchto parametrů

const u8* src	ukazatel na zdrojový komprimovaný obrázek
const u16* pal	ukazatel na paletu barev zdrojového komprimovaného obrázku
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku

DrawImgRle(const u8* src, int xd, int yd, int w, int h);

- vykreslí celý 4bitový nebo 8bitový obrázek v RLE formátu dle těchto parametrů

const u8* src	ukazatel na zdrojový komprimovaný obrázek
int xd	počáteční pozice vykreslování obrázku na displeji na ose x
int yd	počáteční pozice vykreslování obrázku na displeji na ose y
int w	šířka oblasti vykreslení obrázku
int h	výška oblasti vykreslení obrázku

5.6 Příkazy pro práci se zvukovými soubory

Pro práci se zvukovými soubory je využíván hlavičkový soubor `lib_pwm_snd.h`, který nalezneme ve složce `_lib/inc` pro [PicoLibSDK](#)

Pro zvukový výstup PWM je nutné, aby zvukový soubor byl ve formátu 8bit PCM se vzorkovací frekvencí 22 050 Hz.

PWMSndInit();

- inicializuje zvukový PWM výstup

PWMSndTerm();

- ukončení inicializace zvukového PWM výstupu

StopSoundChan(u8 chan);

- zastavení přehrávání zvukového souboru na specifikovaném zvukovém kanálu

u8 chan	definuje číslo kanálu
---------	-----------------------

StopSound();

- zastavení přehrávání zvukového souboru na aktuálním zvukovém kanálu

StopAllSound();

- zastavení přehrávání všech zvukových souborů na všech kanálech

PlaySoundChan(u8 chan, const u8* snd, int len, Bool rep, float speed, float volume, u8 form, int ext);

- přehrávání zvukového souboru na specifikovaném kanálu

u8 chan	definuje číslo kanálu
const u8* snd	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání pro formát zvukového souboru ADPCM je nutno nastavit 2 x počet byte
Bool rep	definuje, zda se přehrávání zvukového souboru bude opakovat či nikoliv hodnota True definuje opakování přehrávání
float speed	definuje rychlost přehrávání hodnota 1 definuje normální rychlost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1
float volume	definuje hlasitost přehrávání hodnota 1 definuje normální hlasitost; nižší hodnota hlasitost sníží, vyšší hodnota hlasitost zvýší
u8 form	definuje formát zvukového souboru
in text	definuje rozšířená data formátu pro formát zvukového souboru ADPCM je nutné nastavit hodnotu na počet vzorků na blok

PlaySound(const u8* snd, int len);

- přehrávání zvukového souboru

const u8	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

PLAYSOUND(snd)

- přehrávání zvukového souboru

snd	ukazatel na zvuková data nutno provést definici zvukového souboru příkazem <i>const u8 <ukazatel na zvuková data> [počet vzorků];</i>
-----	---

PlaySoundRep(const u8* snd, int len);

- přehrávání zvukového souboru v smyčce
- ukončení přehrávání se provede příkazem *StopSound();* nebo *StopAllSound();*

const u8	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

PLAYSOUNDREP(snd)

- přehrávání zvukového souboru v smyčce
- ukončení přehrávání se provede příkazem *StopSound();* nebo *StopAllSound();*

snd	ukazatel na zvuková data nutno provést definici zvukového souboru příkazem <i>const u8 <ukazatel na zvuková data> [počet vzorků];</i>
-----	---

SpeedSoundChan(u8 chan, float speed);

- nastavení rychlosti přehrávání zvukového souboru na specifikovaném kanálu

u8 chan	definuje číslo kanálu
float speed	definuje rychlost přehrávání hodnota 1 definuje normální rychlost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1

SpeedSound(float speed);

- nastavení rychlosti přehrávání zvukového souboru pro všechny kanály

float speed	definuje rychlost přehrávání hodnota 1 definuje normální rychlost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1
-------------	---

VolumeSoundChan(u8 chan, float volume);

- nastavení hlasitosti přehrávání zvukového souboru na specifikovaném kanálu

u8 chan	definuje číslo kanálu
float volume	definuje hlasitost přehrávání hodnota 1 definuje normální hlasitost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1

VolumeSound (float volume);

- nastavení hlasitosti přehrávání zvukového souboru pro všechny kanály

float volume	definuje hlasitost přehrávání hodnota 1 definuje normální hlasitost přehrávání pro formát zvukového souboru ADPCM musí být vždy hodnota nastavena na 1
--------------	---

PlayingSoundChan(u8 chan);

- kontroluje, zda na specifikovaném kanálu probíhá přehrávání zvukového souboru
- návratová hodnota *True* je-li na specifikovaném kanálu přehráván zvukový soubor
- návratová hodnota *False*, není-li na specifikovaném kanálu přehráván zvukový soubor

u8 chan	definuje číslo kanálu
---------	-----------------------

PlayingSound();

- kontroluje, zda na všech kanálech probíhá přehrávání zvukového souboru
- návratová hodnota *True* je-li přehráván zvukový soubor
- návratová hodnota *False*, není-li přehráván zvukový soubor

SetNextSoundChan(u8 chan, const u8* snd, int len);

- nastavení zvukového souboru, jež se přehraje po aktuálně přehrávaném souboru na specifikovaném kanálu

u8 chan	definuje číslo kanálu
const u8* snd	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

SetNextSound(const u8* snd, int len);

- nastavení zvukového souboru, jež se přehraje po aktuálně přehrávaném souboru na všech kanálech

const u8* snd	ukazatel na zvuková data
int len	definuje počet vzorků zvukového souboru a délku jejich přehrávání

GlobalSoundSetOff();

- ukončení všech aktuálně přehrávaných zvukových souborů

GlobalSoundSetOn();

- obnovení přehrávání všech zvukových souborů